RKE2 on Proxmox — Infrastructure Knowledge Base

Production Kubernetes Cluster: Architecture, Operations & Reference

DevOps Team

2026-05-13

Contents

Table of Contents

1. Architecture Overview

Cluster Design Philosophy

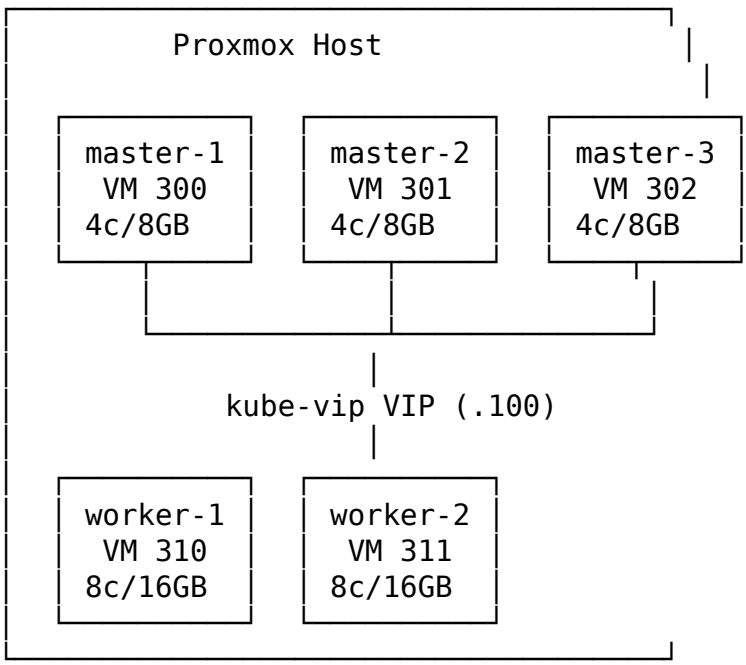
- **High Availability Control Plane** — 3 master nodes with etcd quorum; kube-vip provides a Virtual IP (VIP) so the API server is always reachable regardless of which master is the etcd leader.
- **Separation of concerns** — Masters run only control-plane components; all workloads schedule on workers.
- **Centralized Observability** — No local Grafana, Loki server, or Mimir. Prometheus remote-writes metrics to a centralized Mimir. Grafana Alloy (systemd on each VM) ships pod logs, system logs, and node metrics to central Loki and Mimir. Central Grafana provides all visualization.
- **eBPF-native networking** — Cilium replaces kube-proxy and acts as the IngressController. WireGuard encrypts all node-to-node traffic transparently.
- **GitOps-ready** — All infrastructure is Terraform; all Kubernetes resources are YAML manifests. Secrets are external (ESO + Vault).

Component Summary

Layer	Component	Purpose
Hypervisor	Proxmox VE 7/8	VM host
IaC	Terraform (bpg/proxmox)	VM provisioning
OS	Ubuntu 22.04 cloud-init	Node OS
Cluster	RKE2 v1.29+	Kubernetes distribution
HA	kube-vip	Control-plane VIP + Service LB
CNI	Cilium	Networking, NetworkPolicy, Ingress
Metrics (cluster)	Prometheus (→ central Mimir)	Kubernetes & etcd metrics
Metrics (nodes)	Grafana Alloy systemd (→ central Mimir)	Node-level metrics
Logs	Grafana Alloy systemd (→ central Loki)	Pod logs + system journal
Certificates	cert-manager	TLS automation
Secrets	External Secrets Operator	Vault/cloud secret sync

2. Infrastructure Layout

Node Topology



IP Address Plan

Node	VM ID	Default IP	Role
rke2-master-1	300	192.168.10.101	Init master (etcd bootstrap)
rke2-master-2	301	192.168.10.102	Additional master
rke2-master-3	302	192.168.10.103	Additional master
rke2-worker-1	310	192.168.10.111	Worker
rke2-worker-2	311	192.168.10.112	Worker
Control Plane VIP	—	192.168.10.100	kube-vip (HA API server)
Ingress VIP	—	192.168.10.200	kube-vip (Cilium IngressCon-troller)

All IPs are Terraform variables. Change them in `terraform/proxmox/terraform.tfvars`.

Disk Layout Per Node

Master nodes (2 disks):

Disk	Device	Size	Filesystem	Mount	Purpose
OS disk	`/dev/vda`	50 GB	ext4	`/`	Ubuntu OS + RKE2 binaries
etcd disk	`/dev/vdb`	20 GB	ext4	`/var/lib/rancher/rke2/server/db`	etcd data — dedicated disk prevents I/O starvation

Worker nodes (2 disks):

Disk	Device	Size	Filesystem	Mount	Purpose
OS disk	`/dev/vda`	100 GB	ext4	`/`	Ubuntu OS + RKE2 agent
Data disk	`/dev/vdb`	200 GB	XFS	`/var/lib/rancher`	Container images + PVC volumes

Network CIDRs

Network	CIDR	Notes
Node network	`192.168.10.0/24`	Physical VMs
Pod CIDR	`10.42.0.0/16`	Kubernetes pod IPs
Service CIDR	`10.43.0.0/16`	ClusterIP service IPs
Cluster DNS	`10.43.0.10`	CoreDNS

3. Terraform Modules Reference

Directory Structure

```
terraform/
├── proxmox/                    # VM provisioning
│   ├── provider.tf             # bpg/proxmox provider config
│   ├── main.tf                 # Master + worker module calls
│   ├── variables.tf            # All input variables
│   ├── outputs.tf              # IPs, hostnames, SSH key
│   ├── terraform.tfvars.example
│   ├── modules/
│   │   ├── master_node/        # Single master VM
│   │   └── worker_node/        # Single worker VM
│   └── templates/              # cloud-init, RKE2 configs, inventory
│
└── observability/              # Monitoring stack
    ├── main.tf                 # Module orchestration
    ├── variables.tf            # All variables
    ├── terraform.tfvars.example
    └── modules/
        ├── prometheus/         # kube-prometheus-stack Helm release
      └── prometheus/       # kube-prometheus-stack (Alloy is on-VM, not here)
```

Key Terraform Variables

Proxmox (`terraform/proxmox/terraform.tfvars`)

Variable	Default	Description
`proxmox_api_url`	—	`https://<host>:8006/api2/json`
`proxmox_node`	—	Proxmox node name (e.g. pve)
`proxmox_tls_insecure`	`false`	Skip TLS check (dev only)
`master_count`	3	Must be odd (1, 3, 5)
`master_cpu_cores`	4	CPU cores per master
`master_memory_mb`	8192	RAM per master (MB)
`master_disk_size_gb`	50	OS disk size
`master_etcd_disk_size_gb`	20	Dedicated etcd disk
`worker_count`	2	Scale by incrementing
`worker_cpu_cores`	8	CPU cores per worker
`worker_memory_mb`	16384	RAM per worker (MB)
`worker_data_disk_size_gb`	200	Container storage disk
`control_plane_vip`	—	Free IP for kube-vip
`rke2_cni`	`cilium`	CNI plugin

Observability (`terraform/observability/terraform.tfvars`)

Variable	Required	Description
`central_mimir_url`	**Yes**	Mimir remote-write endpoint
`central_loki_url`	**Yes**	Loki push endpoint
`loki_tenant_id`	No	Defaults to `cluster_name`
`central_mimir_username`	No	Basic auth (if Mimir requires it)
`central_loki_username`	No	Basic auth (if Loki requires it)
`prometheus_retention_days`	`3`	Local buffer (Mimir has long-term)
`prometheus_storage_size`	`20Gi`	Local Prometheus PVC

Sensitive values — always set via environment variables:

```
export TF_VAR_proxmox_password="..."
export TF_VAR_central_mimir_password="..."
export TF_VAR_central_loki_password="..."
export TF_VAR_alertmanager_slack_webhook="..."
```

4. RKE2 Cluster Configuration

Control Plane (Master) Hardening

The RKE2 master config (`rke2-master-config.yaml`) applies these security settings:

API Server flags: - `audit-log-*` — Audit log enabled, 30-day retention - `audit-policy-file` — Custom policy capturing RBAC changes, exec, secret access - `anonymous-auth=false` — No unauthenticated API access - `tls-min-version=VersionTLS12` — TLS 1.2+ only - `enable-admission-plugins=NodeRestriction,PodSecurity` — PSS enforced at admission

etcd tuning: - Heartbeat: `250ms`, election timeout: `5000ms` - Quota: `8 GB`, auto-compaction every 8h - Dedicated disk at `/var/lib/rancher/rke2/server/db`

etcd Snapshots (automatic): - Schedule: every 6 hours (`0 */6 * * *`) - Retention: 10 snapshots - Location: `/var/lib/rancher/rke2/server/db/snapshots/`

Worker (Agent) Configuration

- Max pods: `110`
- Eviction thresholds: memory `<500Mi` hard, `<1Gi` soft
- System reserved: `CPU 200m`, `RAM 512Mi`
- `protect-kernel-defaults=true`
- `read-only-port=0` (kubelet API not exposed)

kube-vip (HA Control Plane)

kube-vip runs as a DaemonSet on all master nodes. It uses ARP to advertise the VIP, so any master can hold it. During leader failover, VIP migrates within ~2 seconds.

Function	VIP	Port
API server	`<control_plane_vip>`	6443
RKE2 agent join	`<control_plane_vip>`	9345
Ingress traffic	`<ingress_vip>`	80/443

Kernel Tuning (all nodes)

Applied via cloud-init provisioner:

```
vm.swappiness=0                       # No swap
vm.overcommit_memory=1                # Required for etcd
net.ipv4.ip_forward=1                 # Pod routing
net.bridge.bridge-nf-call-iptables=1 # eBPF compatibility
fs.inotify.max_user_watches=524288   # For large deployments
net.core.somaxconn=32768              # High-connection workloads
```

5. Cilium CNI & IngressController

Why Cilium

Feature	NGINX Ingress	Cilium
Data plane	iptables/IPVS	eBPF (kernel-native)
kube-proxy	Required	Replaced
Ingress	Via controller pod	Built-in
Network encryption	Not included	WireGuard (automatic)
Network observability	Not included	Hubble (per-flow)
Gateway API	Not supported	Native
Performance	Moderate	High (no userspace hops)

Cilium Configuration Summary

Configured via `rke2/configs/rke2-cilium-config.yaml` (HelmChartConfig). Deployed by `install-master.sh` before RKE2 starts.

Feature	Setting
kube-proxy replacement	kubeProxyReplacement: true
IngressController	enabled, mode: shared, class: cilium
Gateway API	enabled (HTTPRoute, GRPCRoute, TCPRoute)
Hubble	enabled (relay + UI + metrics)
Node encryption	WireGuard, nodeEncryption: true
IPAM	kubernetes (uses node CIDR)
Prometheus metrics	ServiceMonitor enabled → scraped → Mimir

Using Cilium Ingress

Standard Ingress (Kubernetes API):

```yaml
spec:
  ingressClassName: cilium
```

Gateway API (recommended for new applications):

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: my-app
spec:
  parentRefs:
    - name: cilium-gateway
      namespace: kube-system
  hostnames: ["my-app.yourdomain.com"]
  rules:
```

```
    - backendRefs:
        - name: my-app-svc
          port: 80
```

Hubble — Network Observability

Hubble gives you per-flow visibility without any application changes.

```
# Install Hubble CLI
export HUBBLE_VERSION=$(curl -s https://raw.githubusercontent.com/cilium/hubble/maste
curl -L --fail --remote-name-all \
  "https://github.com/cilium/hubble/releases/download/${HUBBLE_VERSION}/hubble-linux-
tar xzvf hubble-linux-amd64.tar.gz
sudo mv hubble /usr/local/bin/

# Port-forward and observe flows
kubectl port-forward -n kube-system svc/hubble-relay 4245:80 &
hubble observe --namespace production --follow
hubble observe --protocol http --verdict DROPPED
```

Hubble metrics appear in central Grafana under the `cluster="rke2-prod"` filter.

6. RBAC — Team Roles & Permissions

Role Matrix

Role	Cluster Scope	Namespaces	Exec	Secrets	RBAC Write
`senior-devops`	Full read/write	All	Yes (all)	Yes	**No**
`junior-devops`	Workloads only	All	Yes (non-prod)	**No**	**No**
`developer`	Namespace list only	`development` (full), `staging` (read)	Yes (dev)	**No**	**No**
`read-only-auditor`	Full read	All	**No**	Metadata only	**No**

Kubeconfig Distribution

Generated by `rbac/scripts/generate-kubeconfigs.sh`. Each kubeconfig uses a **client certificate** with the group name embedded in the 0= (Organization) field — matching the RBAC `ClusterRoleBinding` subjects.

File	Group	Certificate expiry
`kubeconfig-senior-devops-user.yaml`	`senior-devops`	365 days
`kubeconfig-junior-devops-user.yaml`	`junior-devops`	180 days
`kubeconfig-developer-user.yaml`	`developers`	180 days
`kubeconfig-auditor-user.yaml`	`read-only-auditors`	90 days

> **Rotation:** Re-run `generate-kubeconfigs.sh` before expiry. Distribute via encrypted channel — never email or git.

Namespace Pod Security Standards

Namespace	PSS Level	Notes
`production`	`restricted`	No privilege escalation, read-only root FS
`staging`	`baseline`	No host namespace access
`development`	`baseline`	Warn on `restricted` violations

Namespace	PSS Level	Notes
`monitoring`	`privileged`	Prometheus needs host-level access for etcd metrics
`cert-manager`	`restricted`	Hardened

7. Security Hardening

Network Policies (Zero-Trust)

Default deny-all applied to `production` and `staging`. Explicit allow rules:

```
production/staging namespace:
  ├── Deny all ingress + egress (default)
  ├── Allow egress: DNS port 53
  ├── Allow ingress: from kube-system (Cilium IngressController)
  └── Allow ingress: from monitoring namespace (Prometheus scrape)

monitoring namespace:
  ├── Prometheus: egress to all namespaces (for scraping)
  ├── Prometheus: ingress from Grafana (port 9090)
  └── Prometheus: ingress from Alertmanager (port 9090)

development namespace:
  ├── Deny all ingress
  └── Allow all egress (developers need package/API access)
```

Secrets Management

External Secrets Operator (ESO) syncs secrets from Vault into Kubernetes: - Secrets never stored in git or Terraform state - Rotation in Vault propagates automatically (configurable refresh interval) - `ClusterSecretStore` references your Vault backend

Configure your Vault URL in `security/secrets/external-secrets-operator.yaml`:

```yaml
spec:
  provider:
    vault:
      server: "https://vault.YOUR_DOMAIN:8200"
```

TLS — cert-manager ClusterIssuers

Issuer Name	Type	Use For
`letsencrypt-prod`	ACME HTTP-01	Public domains
`letsencrypt-staging`	ACME (test)	Testing (no rate limits)
`cluster-ca-issuer`	Internal CA	Cluster-internal services

Usage in Ingress:

```yaml
annotations:
  cert-manager.io/cluster-issuer: "letsencrypt-prod"
```

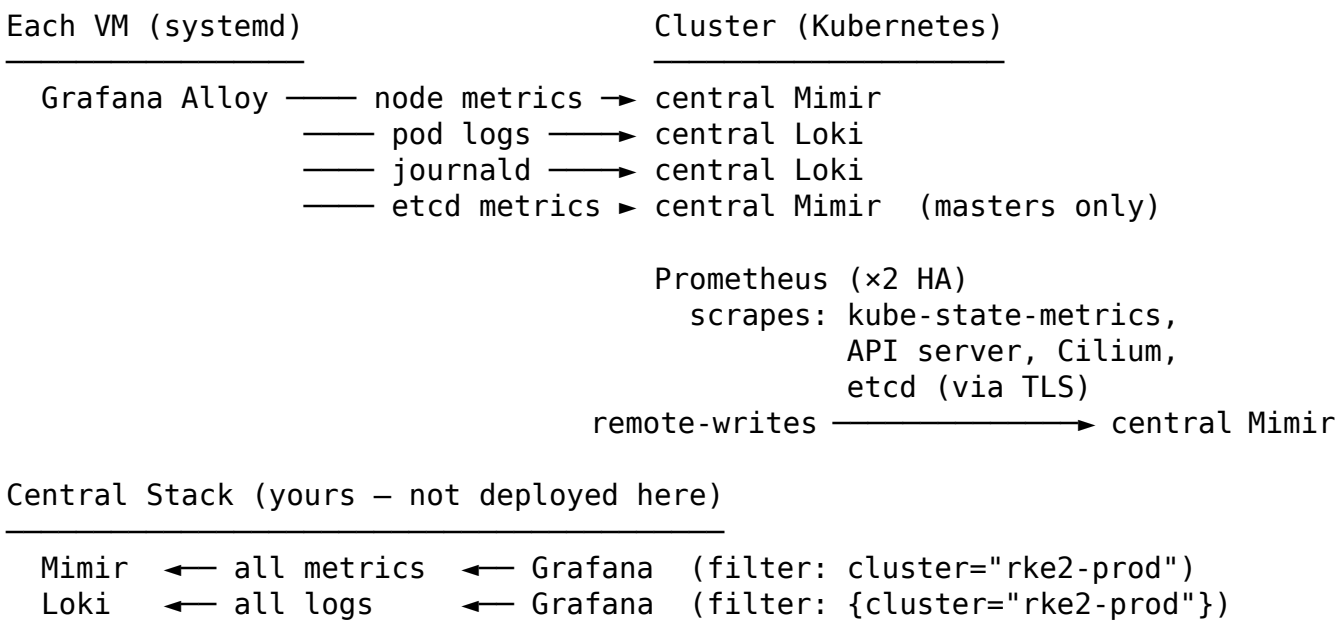
API Server Audit Policy

Captures: - **RequestResponse level:** All RBAC changes (ClusterRoles, Bindings) - **Request level:** `pods/exec`, `pods/attach`, PVC changes, all writes - **Metadata level:** Secret access (no data logged), all reads - **Dropped:** Health check noise, kubelet reads, leader elections

Logs written to `/var/lib/rancher/rke2/server/logs/audit.log` on each master. Forwarded to central Loki by Grafana Alloy reading journald on each master node.

8. Observability Stack

Architecture (Centralized)



What Runs In This Cluster

Component	Helm Chart	Namespace
Prometheus (×2 HA)	kube-prometheus-stack	`monitoring`
Alertmanager (×2)	kube-prometheus-stack	`monitoring`
kube-state-metrics	kube-prometheus-stack	`monitoring`

Intentionally disabled in kube-prometheus-stack: - `grafana.enabled: false` — use your central Grafana - `nodeExporter.enabled: false` — Alloy's `prometheus.exporter.unix` replaces it

What Runs On Each VM (systemd — NOT Kubernetes)

Grafana Alloy is installed as a `systemd` service on every VM by `install-alloy.sh` (called from `install-master.sh` and `install-worker.sh`).

Config: `/etc/alloy/config.alloy` **Logs:** `journalctl -u alloy`

Alloy Component	What It Collects	Destination
`loki.source.file`	Pod logs (`/var/log/pods/*/*/*.log`)	central Loki
`loki.source.journal`	systemd / RKE2 journal logs	central Loki

Alloy Component	What It Collects	Destination
`prometheus.exporter.unix`	Node CPU/mem/disk/net metrics	central Mimir
`prometheus.scrape` (etcd)	etcd metrics port 2381 via TLS	central Mimir (masters only)

All telemetry carries labels: `cluster`, `node`, `environment`, `role`.

Log Labels (every pod log line)

```
cluster="rke2-prod"          ← filter by cluster in Grafana
namespace="production"
pod="my-app-abc123"
container="my-app"
node="rke2-worker-1"
level="info"                 ← extracted from JSON logs
trace_id="abc..."            ← extracted for Tempo correlation
```

Prometheus Remote-Write to Mimir

```yaml
remoteWrite:
  - url: "<central_mimir_url>"
    name: central-mimir
    # WAL buffers up to 2 hours of data if Mimir is unreachable
    queueConfig:
      maxSamplesPerSend: 10000
      capacity: 10000
      maxShards: 30
```

All metrics carry `cluster="rke2-prod"` and `environment="production"` labels.

Alertmanager Routing

```
All alerts
├── severity=critical/page  → PagerDuty
├── severity=warning        → Slack (#alerts-rke2-prod)
├── alertname=Watchdog      → null (silence)
└── default                 → Email
```

Inhibition rule: A `critical` alert suppresses matching `warning` alerts for the same `alertname` + `namespace`.

Recommended Grafana Dashboards (import by ID)

Dashboard	ID	Data Source	Filter
Kubernetes Cluster Overview	7249	Mimir	`cluster="rke2-prod"`

Dashboard	ID	Data Source	Filter
Node Exporter Full	1860	Mimir	`cluster="rke2-prod"` (works with Alloy)
etcd	3070	Mimir	`cluster="rke2-prod"`
Loki Logs	13639	Loki	`{cluster="rke2-prod"}`
RKE2 Cluster	16450	Mimir	`cluster="rke2-prod"`
Cilium / Hubble	16611	Mimir	`cluster="rke2-prod"`

---

9. Deployment Automation

Prerequisites (one-time setup)

- ☐ Proxmox API token created for `terraform@pve`
- ☐ Ubuntu 22.04 cloud-init template created (VM ID set in `vm_template_id`)
- ☐ `terraform/proxmox/terraform.tfvars` filled in
- ☐ `terraform/observability/terraform.tfvars` filled in
- ☐ Required tools installed: `terraform`, `kubectl`, `helm`, `openssl`, `jq`

Single-Command Deployment

```
# Set sensitive values
export TF_VAR_proxmox_password="your-proxmox-password"
export TF_VAR_central_mimir_password="your-mimir-password"   # leave empty if no auth
export TF_VAR_central_loki_password="your-loki-password"    # leave empty if no auth
export TF_VAR_alertmanager_slack_webhook="https://hooks.slack.com/..."

# Run full deployment
chmod +x scripts/deploy.sh
./scripts/deploy.sh
```

Deploy Script Phases

Flag	Example	Effect
(none)	`./scripts/deploy.sh`	Full Phase 2–6
`--from phaseN`	`--from phase4`	Resume from phase N
`--only phaseN`	`--only phase5`	Run one phase only
`--dry-run`	`--dry-run`	Validate, no changes

Estimated Deployment Time

Phase	Duration
Phase 2 — VM provisioning	~8–12 min
Phase 3 — RKE2 installation	~12–18 min
Phase 4 — Security stack	~4–6 min
Phase 5 — Observability	~5–8 min
Phase 6 — Cilium verification	~1 min
Total	**~30–45 min**

Generated Secrets (in `.secrets/` — never commit)

File	Contents
`cluster_id_ed25519`	SSH private key for VM access
`rke2-cluster-token`	RKE2 join token
`kubeconfig-admin.yaml`	Admin kubeconfig (full cluster access)
`tf-outputs.json`	Terraform output values
`certs/<user>/`	Per-role certificate + key

---

10. Scaling Procedures

Add Worker Nodes (zero downtime)

```
# 1. Edit terraform/proxmox/terraform.tfvars
worker_count = 3                          # was 2
worker_ip_addresses = [
  "192.168.10.111",
  "192.168.10.112",
  "192.168.10.113",                       # new
]

# 2. Apply (creates only the new VM)
cd terraform/proxmox && terraform apply

# 3. Join new worker to cluster
./rke2/scripts/install-worker.sh          # idempotent — skips existing nodes

# 4. Verify
kubectl get nodes -o wide
```

Scale Worker CPU/RAM

```
# 1. Drain node
kubectl drain rke2-worker-2 --ignore-daemonsets --delete-emptydir-data

# 2. Update variables and apply to specific module
#    worker_cpu_cores = 12
#    worker_memory_mb = 32768
terraform apply -target='module.worker_nodes[1]'

# 3. Proxmox reboots VM with new resources
# 4. Uncordon
kubectl uncordon rke2-worker-2
```

Add Masters (3 → 5, never 3 → 4)

```
# etcd quorum requires odd number. 3→4 breaks quorum.
master_count = 5
master_ip_addresses = [...existing 3..., "192.168.10.104", "192.168.10.105"]

terraform apply -target='module.master_nodes[3]' -target='module.master_nodes[4]'
./rke2/scripts/install-master.sh          # joins new masters to existing etcd
```

Expand PVC Storage (online, no restart)

```
# StorageClass must have allowVolumeExpansion: true
kubectl patch pvc <pvc-name> -n monitoring \
  -p '{"spec":{"resources":{"requests":{"storage":"100Gi"}}}}'
```

11. Backup & Recovery

Backup Strategy

What	How	Schedule	Location
etcd data	Automatic RKE2 snapshots	Every 6h	`/var/lib/rancher/rke2/server/db/`
etcd data	Offsite copy	Daily cron	S3 / NFS
PVC data	Velero + snapshots	Daily	S3
VM state	Proxmox VM snapshots	Before upgrades	Proxmox storage

Manual etcd Snapshot

```
ssh ubuntu@192.168.10.101
sudo /var/lib/rancher/rke2/bin/rke2 etcd-snapshot save \
  --name manual-$(date +%Y%m%d-%H%M%S)

# List snapshots
sudo /var/lib/rancher/rke2/bin/rke2 etcd-snapshot list
```

etcd Restore

```
# STOP RKE2 on ALL nodes first
sudo systemctl stop rke2-server    # all masters
sudo systemctl stop rke2-agent     # all workers

# Restore on init master only
sudo /var/lib/rancher/rke2/bin/rke2 etcd-snapshot restore \
  --cluster-reset \
  --cluster-reset-restore-path=/var/lib/rancher/rke2/server/db/snapshots/<name>

# Start init master, verify, then start others
sudo systemctl start rke2-server
```

Velero Application Backup

```
# Backup a namespace
velero backup create prod-backup-$(date +%Y%m%d) \
  --include-namespaces production \
  --snapshot-volumes
```

```
# Restore
velero restore create --from-backup prod-backup-20260513

# Schedule (daily 2am, 30-day retention)
velero schedule create daily \
  --schedule="0 2 * * *" \
  --ttl 720h \
  --include-namespaces production,staging
```

12. Upgrade Procedures

RKE2 Upgrade (rolling, zero downtime)

```
TARGET="v1.30.1+rke2r1"

# Step 1: Create backup
sudo /var/lib/rancher/rke2/bin/rke2 etcd-snapshot save \
  --name pre-upgrade-$(date +%Y%m%d)

# Step 2: Upgrade init master
ssh ubuntu@192.168.10.101
curl -sfL https://get.rke2.io | \
  INSTALL_RKE2_VERSION="$TARGET" INSTALL_RKE2_TYPE="server" sh -
sudo systemctl restart rke2-server
# Wait 2-3 minutes for Ready

# Step 3: Upgrade remaining masters (one at a time)
for IP in 192.168.10.102 192.168.10.103; do
  ssh ubuntu@$IP "curl -sfL https://get.rke2.io | \
    INSTALL_RKE2_VERSION='$TARGET' INSTALL_RKE2_TYPE='server' sh - && \
    sudo systemctl restart rke2-server"
  sleep 60
done

# Step 4: Upgrade workers (drain → upgrade → uncordon)
for NODE in rke2-worker-1 rke2-worker-2; do
  kubectl drain $NODE --ignore-daemonsets --delete-emptydir-data
  IP=$(kubectl get node $NODE -o jsonpath='{.status.addresses[?(@.type=="InternalIP")
  ssh ubuntu@$IP "curl -sfL https://get.rke2.io | \
    INSTALL_RKE2_VERSION='$TARGET' INSTALL_RKE2_TYPE='agent' sh - && \
    sudo systemctl restart rke2-agent"
  sleep 60
  kubectl uncordon $NODE
done
```

Observability Stack Upgrade

```
# Update chart version in terraform/observability/modules/prometheus/main.tf
# version = "59.0.0"  (was 58.2.2)

terraform -chdir=terraform/observability apply \
  -target='module.prometheus_stack'
```

> Always upgrade one component at a time. Verify metrics/logs are flowing before continuing.

13. Troubleshooting Quick Reference

Cluster Health Checks

```
# Node status
kubectl get nodes -o wide

# All pods not running
kubectl get pods -A | grep -v "Running\|Completed"

# Resource usage
kubectl top nodes
kubectl top pods -A --sort-by=memory | head -20

# Events (sorted)
kubectl get events -A --sort-by='.lastTimestamp' | tail -20
```

etcd Health

```
ETCD_POD=$(kubectl -n kube-system get pod -l component=etcd -o name | head -1)
kubectl -n kube-system exec $ETCD_POD -- \
  etcdctl endpoint health --cluster \
  --cacert=/var/lib/rancher/rke2/server/tls/etcd/server-ca.crt \
  --cert=/var/lib/rancher/rke2/server/tls/etcd/server.crt \
  --key=/var/lib/rancher/rke2/server/tls/etcd/server.key
```

Common Issues & Fixes

Symptom	Likely Cause	Fix
Node NotReady	kubelet crash / disk pressure	`journalctl -u rke2-server -n 50`; check `df -h`
etcd leader election failing	Slow disk I/O	Check `etcd_disk_wal_fsync_du` in Mimir
Pod stuck Pending	Insufficient resources / PVC not bound	`kubectl describe pod` + `kubectl describe node`
Prometheus not scraping	ServiceMonitor label mismatch	Check `kubectl get service-monitor -A` labels

Symptom	Likely Cause	Fix
Logs not in Loki	Alloy systemd failing	`ssh <node> "journalctl -u alloy -n 50"`
Ingress not responding	Cilium IngressController starting	`kubectl get ingressclass cilium`; restart Cilium pod
VIP unreachable	kube-vip leader down	`kubectl rollout restart ds/kube-vip-ds -n kube-system`
Certificate expired	Cert not rotated	`./rbac/scripts/generate-kubeconfigs.sh`
`terraform apply` 400 Bad Request	VM ID conflict	Run `qm list` on Proxmox host

Useful One-liners

```
# All unhealthy pods
kubectl get pods -A --field-selector=status.phase!=Running,status.phase!=Succeeded

# Network test between pods
kubectl run nettest --image=nicolaka/netshoot --rm -it -- bash

# Decode a secret value
kubectl get secret my-secret -n production -o jsonpath='{.data.password}' | base64 -d

# Force delete a stuck pod
kubectl delete pod <name> -n <ns> --force --grace-period=0

# Check Cilium connectivity
kubectl exec -n kube-system ds/cilium -- cilium connectivity test
```

14. Variable Reference

Environment Variables Required Before `deploy.sh`

```
# Required
export TF_VAR_proxmox_password=""            # Proxmox API password

# Recommended
export TF_VAR_central_mimir_password=""      # Mimir basic auth password
export TF_VAR_central_loki_password=""       # Loki basic auth password
# Alert routing (at least one)
export TF_VAR_alertmanager_slack_webhook="" # Slack webhook URL
export TF_VAR_alertmanager_pagerduty_key="" # PagerDuty integration key
```

Files to Fill In Before Running

File	What to Fill
`terraform/proxmox/terraform.tfvars`	Proxmox IP, node, network, node IPs, VIPs
`terraform/observability/terraform.tfvars`	Mimir URL, Loki URL, tenant ID
`rke2/configs/rke2-cilium-config.yaml`	`loadBalancerIP` for ingress VIP
`security/tls/cluster-issuer.yaml`	Your email for Let's Encrypt
`security/secrets/external-secrets-operator.yaml`	Vault URL

Port Reference

Port	Protocol	Component	Purpose
6443	TCP	kube-apiserver	Kubernetes API
9345	TCP	RKE2	Agent/server join
2379-2380	TCP	etcd	Peer + client communication
10250	TCP	kubelet	API + metrics
9962	TCP	Cilium	Prometheus metrics
4244	TCP	Hubble	Relay
9090	TCP	Prometheus	Query API
12345	TCP	Grafana Alloy	Health / metrics endpoint (local only)

Appendix — Project File Tree

```
infra-setup/
├── scripts/
│   └── deploy.sh                  ← Single-command deployment (Phases 2–6)
├── terraform/
│   ├── proxmox/                   ← VM provisioning
│   │   ├── provider.tf
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   ├── outputs.tf
│   │   ├── terraform.tfvars.example  ← FILL THIS IN
│   │   ├── modules/
│   │   │   ├── master_node/
│   │   │   └── worker_node/
│   │   └── templates/
│   └── observability/             ← Monitoring stack
│       ├── main.tf
│       ├── variables.tf
│       ├── terraform.tfvars.example  ← FILL THIS IN
│       └── modules/
│           └── prometheus/
├── rke2/
│   ├── scripts/
│   │   ├── install-master.sh
│   │   ├── install-worker.sh
│   │   └── install-alloy.sh      ← installs Grafana Alloy as systemd on each VM
│   └── configs/
│       ├── audit-policy.yaml
│       ├── alloy-config.alloy.tpl  ← Alloy River config template
│       └── rke2-cilium-config.yaml ← SET loadBalancerIP
├── rbac/
│   ├── 00-namespace-setup.yaml
│   ├── 01-senior-devops.yaml
│   ├── 02-junior-devops.yaml
│   ├── 03-developer.yaml
│   ├── 04-read-only-auditor.yaml
│   └── scripts/generate-kubeconfigs.sh
├── security/
│   ├── network-policies/
│   ├── secrets/
│   │   └── external-secrets-operator.yaml  ← SET Vault URL
│   └── tls/
│       └── cluster-issuer.yaml             ← SET your email
└── docs/
    ├── implementation-guide.md
    ├── scaling-procedures.md
```

```
├── backup-and-upgrade.md
└── troubleshooting.md
```

Generated: 2026-05-13 | Cluster: rke2-prod | Managed by Terraform